

CSP-Bender Problem

Formulation, Solution in AMPL, and Comparison with Heuristic Methods

Liliu Martinez

Enrique Takiguchi

Contents

1	Introduction	2
2	Problem Description and Formulation	2
2.1	Basic definitions	2
2.2	CSP Formulation	2
2.3	Benders decomposition	3
2.3.1	Subproblem	4
2.3.2	Benders Cuts	6
2.3.3	Master Problem	6
3	Datasets	7
3.1	small	7
3.2	example_2D	7
3.3	targus	8
4	Implementation Details	9
4.1	AMPL models	9
4.1.1	Data params	9
4.1.2	Subproblem	10
4.2	AMPL execution script	10
4.2.1	Initialization	10
4.2.2	Loop	11
5	Computational Experiments and Results	12
5.1	small	12
5.2	Example_2D	13
5.3	targus	13
6	References	14
A	AMPL Files	15

1 Introduction

The Cell Supression Problem (CSP) is a fundamental problem in tabular data protection, as it must be solved to ensure a certain level of confidentiality is applied to specific sensible cells. Tabular data with unsolved CSP takes the risk that an attacker might be able to compute an interval for the values of sensible censured cells which is too small.

Since supressing only sensible cells would not suffice, the CSP seeks to find a balance between the amount of information and the protection of the tables, by defining minimum protection levels and the cost of hiding each cell.

As we will see, the complexity of the raw problem requires an algorithm to solve the problem in an efficient way, for which we will use Benders decomposition.

Objectives:

- Contextualizing the problem
- Formulating the problem
- Implementing CSP in its basic form (MILP) in AMPL
- Implementing CSP with Benders decomposition in AMPL
- Testing the implementations

2 Problem Description and Formulation

2.1 Basic definitions

- **Microdata:** a file V of s individuals and t variables, as a matrix $s \times t$ where V_{ij} is the value of variable j for individual i .
- **Tabular data:** tables generated crossing one ore more categorical variables from microdata. These can be multidimensional tables holding frequency or magnitude information. The table will have n cells which will have to verify m linear constraints. The cells in the table are bounded by an upper and lower limit: $u < a_i < l$
- **Protection levels from the p% rule:**

An attacker might estimate the intervals of a cell $[\bar{a}_p, \underline{a}_p]$, to assert these are not informative enough, we must define some lower and upper protection level bounds to ensure this interval fully contains the interval

$$[\bar{a}_p, \underline{a}_p] \in [a_p - lpl_p, a_p + upl_p] \quad (1)$$

The best guess an auditor can make about a_p is defined by $[\bar{a}_p, \underline{a}_p] \rightarrow$

$$\underline{a}_p = \begin{array}{ll} \min & x_p \\ \text{s.to} & Ax = b \\ & l_i \leq x_i \leq u_i \quad i \in \mathcal{P} \cup \mathcal{S} \\ & x_i = a_i \quad i \notin \mathcal{P} \cup \mathcal{S} \end{array} \quad \bar{a}_p = \begin{array}{ll} \max & x_p \\ \text{s.to} & Ax = b \\ & l_i \leq x_i \leq u_i \quad i \in \mathcal{P} \cup \mathcal{S} \\ & x_i = a_i \quad i \notin \mathcal{P} \cup \mathcal{S} \end{array} \quad (2)$$

2.2 CSP Formulation

The Cell Suppression Problem seeks to hide sensitive (primary, $\mathcal{P}$) cells while minimizing the suppression cost. Hiding only primary cells would not be enough since we could still infer information about them from the rest of cells, thus, a number of secondary cells ($\mathcal{S}$) will also be suppressed.

CSP computes the set $\mathcal{S}$ of secondary cells to be suppressed such that for all $p \in \mathcal{P}$, Equation 1 is satisfied, where $\bar{a}_p, \underline{a}_p$ can be obtained with a minimization problem.

The CSP problem can be formulated as a Mixed-Integer Linear Programming (MILP) problem. The MILP will be a minimization problem in terms of the following variables

- $y_i \in \{0, 1\}, i = 1, \dots, n$: Indicates if a cell i is suppressed (1) or not (0)
- $x^{l,p} \in \mathbb{R}^n$: Lower deviation of the primary cell p from its original value
- $x^{u,p} \in \mathbb{R}^n$: Upper deviation of the primary cell p from its original value

MILP formulation for CSP:

$$\begin{aligned}
& \min \quad \sum_{i=1}^n w_i y_i \\
& \text{s.to} \quad \left. \begin{aligned}
& Ax^{l,p} = 0 \\
& (l_i - a_i)y_i \leq x_i^{l,p} \leq (u_i - a_i)y_i \quad i = 1, \dots, n \\
& x_p^{l,p} \leq -lpl_p \\
& Ax^{u,p} = 0 \\
& (l_i - a_i)y_i \leq x_i^{u,p} \leq (u_i - a_i)y_i \quad i = 1, \dots, n \\
& x_p^{u,p} \geq upl_p
\end{aligned} \right\} \forall p \in \mathcal{P} \\
& y_i \in \{0, 1\} \quad x \geq 0 \quad i = 1, \dots, n,
\end{aligned}$$

The problem with the MILP formulation is that it becomes infeasibly complex as the table grows large.

The problem has:

- n binary variables and $2n\mathcal{P}$ continuous variables
- $2(m + 2n)|\mathcal{P}|$ constraints

2.3 Benders decomposition

Benders decomposition partitions the problem into a master and subproblems, and iterates efficiently between them to find the optimal solution.

- Master problem: focuses on the binary decisions (y_i), which determine whether a cell is published ($y_i = 0$) or suppressed ($y_i = 1$)
- Subproblems: verify if the current suppression pattern chosen by the Master Problem actually provides the required protection levels (lpl and upl) for each primary sensitive cell.

Original Problem

Given the original problem:

$$\begin{aligned}
(P) = \quad & \min \quad c^T x + q^T y \\
& \text{s. to} \quad Tx + Wy = h \\
& x \in X \\
& y \geq 0
\end{aligned}$$

The two following problems are derivated:

- **Master Formulation**

$$\begin{aligned}
(BP_r) = \quad & \min \quad z \\
\text{s. to} \quad & z \geq c^T x + u^{i\top} (h - Tx) & i \in I \subseteq \{1 \dots p\} \\
& v^{j\top} (h - Tx) \leq 0 & j \in J \subseteq \{1 \dots q\} \\
& x \in X.
\end{aligned}$$

- **Subproblem Formulation**

$$\begin{aligned}
(Q_D) = \quad & \max \quad u^T (h - Tx) \\
\text{s. to} \quad & W^T u \leq q \\
& u \in \mathbb{R}^m
\end{aligned}$$

To take advantage of Benders decomposition we first reformulate the original MILP problem to an equivalent of Benders starting problem.

$$\begin{aligned}
\min \quad & \sum_{i=1}^n c_i y_i \\
\text{s. to} \quad & Ax^{l,p} = 0 & \forall p \in \mathcal{P} \\
& Ax^{u,p} = 0 & \forall p \in \mathcal{P} \\
& (l_i - a_i)y_i \leq x_i^{l,p} & i = 1 \dots n, \forall p \in \mathcal{P} \\
& x_i^{l,p} \leq (u_i - a_i)y_i & i = 1 \dots n, \forall p \in \mathcal{P} \\
& (l_i - a_i)y_i \leq x_i^{u,p} & i = 1 \dots n, \forall p \in \mathcal{P} \\
& x_i^{u,p} \leq (u_i - a_i)y_i & i = 1 \dots n, \forall p \in \mathcal{P} \\
& x_p^{l,p} \leq -lp l_p & \forall p \in \mathcal{P} \\
& x_p^{u,p} \geq up l_p & \forall p \in \mathcal{P} \\
& y_i \in \{0, 1\}, \quad x, s \geq 0
\end{aligned}$$

Given that y_i belongs to a restricted set and both x and s are only restricted to be positive, we take y_i as the x in Benders and $[x, s]$ as the y .

The Original Problem now becomes:

	$\mathbf{c}^T \mathbf{x}$	$\mathbf{q}^T \mathbf{y}$
Objective function (min)	$\sum_{i=1}^n c_i y_i$	$+0 \sum_{i=1}^n x_i^{l,p} + x_i^{u,p}$

2.3.1 Subproblem

With this formulation we can now derive the Subproblem and Master Problem for Benders decomposition. The subproblem will have to find the maximum deviation that an attacker can achieve for each primary cell p while respecting the constraints of the table and it is within the protection levels. Primal subproblem Q for each primary cell p :

Constraint Type	Tx	+Wy	=h
Table Conservation	0	$+Ax^{l,p}$ $+Ax^{u,p}$	0
Lower Capacity	$(l-a)y$ $(l-a)y$	$-x^{l,p}$ $-x^{u,p}$	≤ 0
Upper Capacity	$(a-u)y$ $(a-u)y$	$+x^{l,p}$ $+x^{u,p}$	≤ 0
Protection Levels	0	$x^{l,p}$ $x^{u,p}$	$\leq -lpl$ $\geq upl$

	$x \in X$	$y \geq 0$
Domain Constraints	$y \in \{0, 1\}$	$x^l, x^u \geq 0$

- Lower protection level subproblem:

$$\begin{aligned}
(Q) = \quad & \min \quad x_p \\
\text{s. to} \quad & Ax = b[\lambda] \\
& (l_i - a_i)\bar{y}_i \leq x_i[\mu^l] \\
& x_i \leq (u_i - a_i)\bar{y}_i[\mu^u] \\
& -x_p \leq lpl_p[\gamma]
\end{aligned}$$

- Upper protection level subproblem:

$$\begin{aligned}
(Q) = \quad & \min \quad x_{p[j]} \\
\text{s. to} \quad & Ax = b[\lambda] \\
& (l_i - a_i)\bar{y}_i \leq x_i[\mu^l] \\
& x_i \leq (u_i - a_i)\bar{y}_i[\mu^u] \\
& x_{p[j]} \leq upl_{p[j]}[\gamma]
\end{aligned}$$

Therefore, if we denote *protection_lv* as either lpl_p or upl_p , and L as the corresponding correct sign, both subproblems can be summarized as:

$$\begin{aligned}
(Q) = \quad & \min \quad x_p \\
\text{s. to} \quad & Ax = b[\lambda] \\
& (l_i - a_i)\bar{y}_i \leq x_i[\mu^l] \\
& x_i \leq (u_i - a_i)\bar{y}_i[\mu^u] \\
& Lx_p \leq \text{protection_lv}l_p
\end{aligned}$$

Given the primary, with $\mathcal{C}$ as the number of constraints in A matrix, we can derive the dual subproblem.

$$\begin{aligned}
(Q_D) = \quad & \max \quad \sum_{c \in \mathcal{C}} \lambda_c b_c + \sum_{i=1}^n ((l_i - a_i)\mu_i^l + (u_i - a_i)\mu_i^u)\bar{y}_i + \gamma \text{protection_lv}l_{p[j]} \\
\text{s. to} \quad & A^T \lambda + \mu^l - \mu^u = e_p \gamma \quad i = 1..n \\
& \mu^l \geq 0, \mu^u \leq 0, \lambda_{free}, \gamma \geq 0
\end{aligned}$$

2.3.2 Benders Cuts

With this changes, we no longer have lower and upper subproblems that gives us different cuts to add to the Master Problem. The cuts to add to the Master Problem will depend on the optimal value of the combined dual subproblem objective such as:

- Original bender cuts:

- Lower Benders cut:

$$-lpl \geq \sum_{i=1}^n ((l_i - a_i)\mu_{ll,i} - (u_i - a_i)\mu_{ul,i})y_i$$

- Upper Benders cut:

$$upl \leq \sum_{i=1}^n (-(l_i - a_i)\mu_{lu,i} + (u_i - a_i)\mu_{uu,i})y_i$$

- New Bender cuts:

$$\begin{aligned} \sum_{i=1}^n ((l_i - a_i)\mu_i^l + (u_i - a_i)\mu_i^u)y_i \quad \forall k \in \mathcal{CUTS} \\ + \gamma_k \text{protection_lvl}_k \leq 0 \end{aligned}$$

2.3.3 Master Problem

The Master Problem will now seek to minimize the suppression cost while respecting the Benders cuts added from previous iterations.

Given:

- $\mathcal{CUTS}$: set of cuts added until iteration r from insecure primary cells found with the lower dual subproblem
- for each cut $k \in \mathcal{CUTS}$, we have $\bar{\mu}_l^{p,k}, \bar{\mu}_u^{p,k}, \bar{\gamma}$ from the solution of the dual subproblem

The Master Problem at iteration r is:

$$\begin{aligned} (BP_r) = \quad & \min \quad \sum_{i=1}^n c_i y_i \\ \text{s. to} \quad & y_p = 1 \quad \forall p \in \mathcal{P} \\ & \sum_{i=1}^n ((l_i - a_i)\bar{\mu}_i^l + (u_i - a_i)\bar{\mu}_i^u)y_i \quad \forall k \in \mathcal{CUTS} \\ & + \bar{\gamma}_k \text{protection_lvl}_k \leq 0 \\ & y_i \in \{0, 1\} \end{aligned}$$

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7
Row 1	300*	8	5	5	5*	38	361
Row 2	8	68*	40	76	29	31	252
Row 3	11	33	20	60	35	44	203
Row 4	7	28	36*	41	22	63	197
Row 5	326	137	101	182	91	176	1013

Table 1: Original Small Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
1	15	20
5	2	4
9	10	6
24	9	3

Table 2: Table of Small Instance Protection Levels for Sensitive Cells

3 Datasets

3.1 small

Original table: In Table 1 we can see the original data table for *small.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Column 7 and Row 5 contain the totals for the rows and columns respectively.

Table Constraints:

Row Constraints:

1. $x_1 + x_2 + x_3 + x_4 + x_5 + x_6 - x_7 = 0$
2. $x_8 + x_9 + x_{10} + x_{11} + x_{12} + x_{13} - x_{14} = 0$
3. $x_{15} + x_{16} + x_{17} + x_{18} + x_{19} + x_{20} - x_{21} = 0$
4. $x_{22} + x_{23} + x_{24} + x_{25} + x_{26} + x_{27} - x_{28} = 0$

Column Constraints:

5. $x_1 + x_8 + x_{15} + x_{22} - x_{29} = 0$
6. $x_2 + x_9 + x_{16} + x_{23} - x_{30} = 0$
7. $x_3 + x_{10} + x_{17} + x_{24} - x_{31} = 0$
8. $x_4 + x_{11} + x_{18} + x_{25} - x_{32} = 0$
9. $x_5 + x_{12} + x_{19} + x_{26} - x_{33} = 0$
10. $x_6 + x_{13} + x_{20} + x_{27} - x_{34} = 0$

3.2 example_2D

In Table 3 we can see the original data table for *example_2D.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Column 1 and Row 1 represent the totals for their respective dimensions.

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6
Row 1	3220	84	632	930	843	731
Row 2	1529	3	336	309	484	397
Row 3	484	25	3	393*	48	15
Row 4	392	1	2	137*	145	107
Row 5	815	55	291*	91	166	212*

Table 3: Original Example_2D Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
16	40	30
22	14	14
27	15	30
30	21	21

Table 4: Table of Example_2D Instance Protection Levels for Sensitive Cells

3.3 targus

In Table 5 we can see the original data table for *targus.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Due to the large values in the table, we have used commas to separate thousands. Due to the row-wise size of the table, we have transposed the original table for better visualization. Row 1 contains the totals for the columns.

	Row 1	Row 2	Row 3	Row 4	Row 5	Row 6	Row 7	Row 8	Row 9
Col 1	16847261.84	20*	25	2711808	2320534	2505042.58	2799074.26	6510758	0
Col 2	4373279	5*	5*	719049	659680	688962	756529	1549049	0
Col 3	1986129	5*	5*	398062	348039	354711	418778	466529	0
Col 4	1808861	0	0	223990	221332	241913	258233	863393	0
Col 5	578289	0	0	96997	90309	92338	79518	219127	0
Col 6	3703896	15*	5*	642238	515003	534147	620392	1392096	0
Col 7	124336	5*	0	36311	32132	25770	18150	11968*	0
Col 8	526279	0	0	93589	94957	110930	81799	145004	0
Col 9	2234995	10*	5*	345803	251358	251188	303377	1083254	0
Col 10	818286	0	0	166535	136556	146259	217066	151870	0
Col 11	4576115.84	0	0	648972	543570	663896.58	775132.26	1944545	0
Col 12	485326	0	0	63767	75442	87305	59953	198859	0
Col 13	3664559.84	0	0	537911	430851	515019.58	643762.26	1537016	0
Col 14	426230	0	0	47294	37277	61572	71417	208670	0
Col 15	4193971	0	15*	701549	602281	618037	647021	1625068	0
Col 16	2752743	0	15*	488613	392395	363490	402925	1105305	0
Col 17	1441228	0	0	212936	209886	254547	244096	519763	0
Col 18	0	0	0	0	0	0	0	0	0

Table 5: Original Targus Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
19	6	6
20	1.67	1.67
21	1.67	1.67
24	5	5
25	1.67	1.67
27	3.33	3.33
38	1.67	1.67
39	1.67	1.67
42	1.67	1.67
45	1.67	1.67
51	5	5
52	5	5
133	3480	3480

Table 6: Table of Targus Instance Protection Levels for Sensitive Cells

4 Implementation Details

Given the the mathematical formulation of the Benders decomposition from Section 2.3, we have implemented the master problem and subproblems:

4.1 AMPL models

4.1.1 Data params

```

1 param ncells;
2 param npcells;
3 param nconstraints;
4 param nnz;
5
6 set CELLS := 1..ncells;
7 set P_INDEX := 1..npcells;
8 set CONSTR := 1..nconstraints;
9 set NZ := 1..nnz;
10
11 param a {CELLS};
12 param lb {CELLS};
13 param ub {CELLS};
14 param c {CELLS};
15 param is_p {CELLS};
16
17 param p {P_INDEX};      # Maps primary index to cell number
18 param plpl {P_INDEX};   # Lower protection level
19 param pupl {P_INDEX};   # Upper protection level
20
21 param b {CONSTR};
22 param begconst {1..nconstraints+1};
23 param coef {NZ};
24 param xcoef {NZ};
25
26 \end{verbatim*}
27 \subsubsection{Master Problem}
28 \begin{verbatim*}
29 # --- Benders Cut Storage ---
30 param nCUT >= 0 integer;
31 param mu_l {CELLS, 1..nCUT};

```

```

32 param mu_u {CELLS, 1..nCUT};
33 param protection_lvl {1..nCUT};
34 param gamma {1..nCUT};
35
36
37 var y {CELLS} binary;           # Master variables
38
39 minimize Total_Cost: sum {i in CELLS} c[i] * y[i];
40
41 subject to Primary_Must_Be_Suppressed {k in P_INDEX}:
42     y[p[k]] = 1;
43
44 subj to ProtectionCut {k in 1..nCUT}:
45     sum {i in CELLS} (mu_l[i,k]*(lb[i]-a[i])
46     + mu_u[i,k]*(ub[i]-a[i])) * y[i]
47     + gamma[k] * protection_lvl[k]
48     <= 0;

```

4.1.2 Subproblem

```

1
2 param y_fixed {CELLS};
3 param p_index;           # The cell index of the primary being checked
4 param L_or_U;           # -1 for Lower protection (min xp), +1 for Upper (max xp)
5 param PROTECTION_LVL;
6
7 var LAMBDA {CONSTR};     # Dual variables for lower protection
8 var MU_L {CELLS} >= 0;   # Dual variables for lower protection
9 var MU_U {CELLS} <= 0;   # Dual variables for lower protection
10 var GAMMA >= 0;
11
12 maximize Dual_Objective_Function:
13     sum {i in CONSTR} LAMBDA[i]* b[i]
14     + sum {i in CELLS} MU_L[i]*(lb[i]-a[i])*y_fixed[i]
15     + sum {i in CELLS} MU_U[i]*(ub[i]-a[i])*y_fixed[i]
16     + GAMMA * PROTECTION_LVL;
17
18 # Dual Restriction: Ensures dual feasibility relative to cell deviations [cite:
19     584, 596]
19 subj to Dual_Restriction {i in CELLS}:
20     (sum {l in NZ: xcoef[l] == i} coef[l] * (
21         sum {r in CONSTR: begconst[r] <= 1 < begconst[r+1]} LAMBDA[r]
22     ))
23     + MU_L[i] + MU_U[i]
24     + (if i == p_index then GAMMA * L_or_U else 0) == 0;

```

4.2 AMPL execution script

4.2.1 Initialization

```

1
2
3 let nCUT := 0;
4 param violation_found binary default 0;
5 param iteration default 0;
6 param epsilon := 1e-7;
7 param current_dual_val;
8 suffix unbdd OUT;
9
10 set cut_types{1..2} := {-1, 1};
11 # -1 for lower protection, 1 for upper protection

```

```

12
13 problem Master:
14     y,
15     Total_Cost,
16     Primary_Must_Be_Suppressed,
17     ProtectionCut;
18
19 problem Dual_Problem:
20     LAMBDA,
21     GAMMA,
22     MU_L,
23     MU_U,
24     Dual_Objective_Function,
25     Dual_Restriction;

```

4.2.2 Loop

```

1
2 printf "\nStarting Benders Decomposition for CSP\n";
3 printf "=====\n";
4
5 repeat {
6     let iteration := iteration + 1;
7     let violation_found := 0;
8     #if iteration > 5 then break; #debug show only 2 iterations
9
10    # 1. SOLVE MASTER PROBLEM (Slide 75, Step 2)
11    printf "\nIteration %d: Solving Master Problem... \n", iteration;
12    solve Master;
13    printf "Cost: %.2f\n", Total_Cost;
14
15    # Fix the current suppression pattern for the subproblems
16    for {i in CELLS} {
17        let y_fixed[i] := y[i];
18        printf "%d ", y[i];
19    }
20    printf "\n";
21
22    # 2. SUBPROBLEM LOOP
23    for {i in 1..2, cut_type in cut_types[i]} {
24        # Check every primary cell p for both Lower and Upper protection
25        for {j in P_INDEX} {
26            let p_index := p[j];
27            let L_or_U := cut_type;
28            let PROTECTION_LVL := if cut_type == 1 then pupl[j] else plpl[j];
29
30            printf "\n primary cell %d \n", p_index;
31
32            # --- A. Check Lower Protection (Slide 76-77) ---
33            solve Dual_Problem;
34            if Dual_Problem.result = "unbounded" then {
35                let nCUT := nCUT + 1;
36                let gamma[nCUT] := GAMMA.unbdd;
37                let {ii in CELLS} mu_l[ii, nCUT] := MU_L[ii].unbdd;
38                let {ii in CELLS} mu_u[ii, nCUT] := MU_U[ii].unbdd;
39                let protection_lvl[nCUT] := PROTECTION_LVL;
40
41                let violation_found := 1;
42
43                if cut_type == 1 then {
44                    printf "Added a cut for upper protection in primary cell %d\n", p[j];
45                }

```

```

46     else {
47         printf "Added a cut for lower protection in primary cell %d\n", p[j];
48     }
49 }
50 }
51 }
52 # 3. CONVERGENCE CHECK (Slide 79, Step 5)
53 # If no new constraints were added to J, the solution is optimal.
54 if violation_found = 0 then {
55     printf "\nConvergence reached! No violations found.\n";
56     break;
57 }
58 };

```

5 Computational Experiments and Results

5.1 small

Cell ID	Cost	Is Primary
1	1	1
2	1	0
3	1	0
5	1	1
6	1	0
8	1	0
9	1	1
10	1	0
15	1	0
20	1	0
22	1	0
23	1	0
24	1	1
26	1	0

Table 7: Table of Cells to be Suppressed in Small Instance

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7
Row 1	x	x	X	5	X	X	361
Row 2	8	X	X	X	29	31	252
Row 3	X	33	20	60	35	X	203
Row 4	X	X	X	41	X	63	197
Row 5	326	137	101	182	91	176	1013

Table 8: Small Data Table with Suppressing Sensitive Cells

5.2 Example_2D

Cell ID	Cost	Is Primary
3	0	0
6	0	0
13	0	0
16	0	1
19	0	0
22	0	1
27	0	1
30	0	1

Table 9: Table of Cells to be Suppressed in example_2D Instance

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6
Row 1	3220	84	x	930	843	x
Row 2	1529	3	336	309	484	397
Row 3	x	25	3	x	48	15
Row 4	x	1	2	x	145	107
Row 5	815	55	x	91	166	x

Table 10: example_2D Data Table with Suppressing Sensitive Cells

5.3 targus

Cell ID	Cost	Is Primary
19	1	1
20	1	1
21	1	1
24	1	1
25	1	1
27	1	1
37	1	0
38	1	1
39	1	1
42	1	1
45	1	1
51	1	1
52	1	1
78	611	0
87	715	0
99	298	0
106	432	0
133	14	1
134	172	0

Table 11: Table of Cells to be Suppressed in Targus Instance

	Row 1	Row 2	Row 3	Row 4	Row 5	Row 6	Row 7	Row 8	Row 9
Col 1	16847261.84	x	x	2711808	2320534	2505042.58	2799074.26	6510758	0
Col 2	4373279	x	x	719049	659680	688962	756529	1549049	0
Col 3	1986129	x	x	398062	348039	354711	418778	466529	0
Col 4	1808861	0	0	223990	221332	241913	258233	863393	0
Col 5	578289	0	0	96997	90309	92338	79518	219127	0
Col 6	3703896	x	x	642238	x	534147	620392	1392096	0
Col 7	124336	x	0	36311	32132	25770	18150	x	0
Col 8	526279	0	0	93589	94957	110930	81799	x	0
Col 9	2234995	x	x	345803	251358	x	303377	1083254	0
Col 10	818286	0	0	166535	136556	146259	217066	151870	0
Col 11	4576115.84	0	0	648972	543570	663896.58	775132.26	1944545	0
Col 12	485326	0	0	63767	75442	87305	59953	198859	0
Col 13	3664559.84	0	0	537911	430851	515019.58	643762.26	1537016	0
Col 14	426230	0	0	47294	37277	61572	71417	208670	0
Col 15	4193971	0	x	701549	x	618037	647021	1625068	0
Col 16	2752743	0	x	488613	392395	x	402925	1105305	0
Col 17	1441228	0	0	212936	209886	254547	244096	519763	0
Col 18	0	0	0	0	0	0	0	0	0

Table 12: Targus Data Table with Suppressing Sensitive Cells

6 References

A AMPL Files

```
1 param n; # observations
2 param k; # clusters
3 set M:={1..n};
4
5 param d{M,M};
6 var x{M,M} binary;
7
8
9 minimize f: sum{i in M}
10      sum{j in M} d[i,j]*x[i,j];
11
12 subject to one_cluster{i in M}:
13      sum{j in M} x[i,j] = 1;
14
15 subject to k_clusters:
16      sum{j in M} x[j,j] = k;
17
18 subject to cluster_belongs{i in M, j in M}:
19      x[j,j] >= x[i,j];
```

Listing 1: kmedoid.mod

```
1 # Reset enviroment
2 reset;
3
4 # Load model and data
5 model kmedoids.mod;
6 data kmedoids.dat;
7
8 option solver cplex;
9 solve;
10
11 display _solve_elapsed_time;
12
13 printf {i in M, j in M} "%s,%s,%i\n", i, j, x[i,j] > "x_matrix.csv";
```

Listing 2: kmedoid.run

```
1 def generate_dat(N, D, k, filename='AMPL/kmedoid.dat'):
2     with open(filename, 'w') as f:
3         f.write("#Define sets\n")
4         f.write(f"param n := {N};\n")
5         f.write(f"param k := {k};\n")
6
7         f.write("#Define parameters\n")
8         f.write(f"param d: {'\t'.join([str(j+1) for j in range(N)])} :=\n")
9         for i in range(N):
10             f.write(f" {i+1} " + ' '.join(f"{D[i][j]:.6f}" for j in range(N)) +
11                 "\n")
12             f.write(";\n")
13
```

Listing 3: Python function to generate kmedoid.dat